Infrastructure Module Reference

Infrastructure Module Reference I

This section inventories every Layer-1 subdirectory returned by 31 `discover_infrastructure_modules(repo_root)`. File totals use 1304 Python sources across infra + 12,305 infra tests guarding them. Documentation Duality = paired `README.md` + `AGENTS.md`; optional `SKILL.md` manifests feed `python -m infra structure.skills`.

Infrastructure Module Reference II

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	11			build_autoresearch_plan, readiness validation CLI
benchmark	4			Template harness scoring + comparative gates
config	0			Repository defaults + hardened templates
core	193			get_logger, load_config, TemplateError
docker	0			Containerisation scaffolding
doctor	14			Checkout diagnose/fix/undo repairs
documentation	23			FigureManager, build_api_index, generate_markdown_table
fonds	6			—
formal_contracts	5			—
llm	57			Ollama helpers, sanitization, review + translation pipelines
logrotate.d	0			Rotation snippets (documentation-first)
metadata	3			normalized_repository_url, generate_metadata_package
methods	10			build_methods_orchestration_plan, methods-stage contracts + validation
orchestration	14			PipelineRunner, entry point for ./run.sh

Alphabetical summaries

Alphabetical summaries I

Below, `${module_*_python_file_count}` placeholders expand per subdirectory at render-time.

Readiness planner, validation CLI, and report models for AutoResearch-style project promotion (infrastructure/autoresearch/).

infrastructure.benchmark (4 files) I

Template harness scoring and comparative gate helpers exercised in CI smoke paths.

infrastructure/config (non-package subdirectory) I

Repository-wide YAML templates and secure manifests (`.env.template`, hardened defaults referenced by Docker + CLI). `config/` carries no `__init__.py`, so it is a configuration subdirectory rather than an importable package.

Checkpointing, logging, pipeline YAML parsing, telemetry bridges, filesystem helpers, hardened exceptions. Everything else imports logging + error taxonomy from here first.

Checkout diagnose/fix/undo repairs for broken local workspace states.

Pinned images / compose scaffolding for reproducible CI + remote builds.

Figure registries plus glossary tooling feeding manuscript automation.

Typed, machine-checkable manuscript contract layer: frozen block model, evidence-tier policies, monoidal composition, and a fail-closed checker with stable `FORMAL.*` diagnostics.

Resource pool management for curated fonds (tracked reference datasets, bibliographic collections, and evidence corpora). Fonds mirror `projects/templates/` with git-tracked `templates/*` exemplars and sidecar-linked private lifecycle folders.

Ollama integrations, sanitization adapters, templated reviewer flows. **Literature ingestion now lives primarily in `search/literature` + citation helpers in `reference/`.**

Shared publication-metadata leaves: repository-URL normalization (`normalized_repository_url`) and ebook metadata package generation (ONIX, `metadata.json`, EPUB OPF). Consumed by publishing, rendering, documentation, and project layers; extracted from `publishing` by RENDERING-LAYERING-1 so renderers import it without an inverted publishing dependency.

infrastructure.methods (10 files) I

Deterministic methods-orchestration contracts (`MethodStage`, `MethodsOrchestrationPlan`, `MethodsIssue`): builds and validates an ordered methods plan for a research project so the manuscript's "Methods" track stays bound to executable stages.

`python -m infrastructure.orchestration` exposes interactive menus, subprocess wiring for thin shell wrappers (`run.sh`, `secure_run.sh`), and stubs used in CI for menu parsing tests.

Canonical discovery (`discover_projects`) enforcing `src/` + `tests/`, slug validation, nested WIP namespaces.

Readability metrics + Markdown tooling for prose-centric manuscripts / CI gates.

Content-addressed provenance DAG. Records artifact lineage (which run produced which file, from which inputs) as a verifiable graph of artifact/run/source/claim nodes connected by produced/consumed/derived-from/supports/refutes edges. Includes a structured Review system with severity (blocking/major/minor/info) and verdict (refutes/supports). Features a CLI and pipeline integration hooks for automatic lineage recording after every stage.

infrastructure.publishing (83 files) I

Metadata models, APA/BibTeX/MLA formatters, optional Zenodo clients.

infrastructure.reference (16 files) I

Citation/BibTeX parsing + conversion utilities leveraged by manuscripts and retrieval scripts.

Pandoc shim, Unicode/XeLaTeX postprocessors, combined PDF/HTML/slide exporters.

Parses pytest + coverage artefacts for dashboards; pairs with Stage 01 summaries and downstream executive exports.

Seven-stage research workflow (SCOPE→LITERATURE→REASON→DESIGN→COMPUTE→SYNTHE defined as typed ResearchStage data classes with explicit outputs, skills, and template commands per stage. Includes a full PRISMA-adapted literature review prompt and a research workflow prompt referencing template/ infrastructure commands.

Governance rules layer for validating project lifecycle transitions, sidecar sync policies, and pipeline gate orchestration. Rules mirror `projects/templates/` with git-tracked `templates/*` exemplars.

infrastructure.scientific (4 files) I

Stability probing, benchmarking hooks—consumed heavily by optimization exemplars (template_code_project scripts).

Two-tier search architecture: the `literature/` client stack (client.py, backends, caches) powers literature search with arXiv, Crossref, local, and Paperclip backends. `connectors/` exposes the built-in scientific database adapters through a uniform `Connector Registry`; OpenAlex, UniProt, PDB, Semantic Scholar, European PMC, bioRxiv, and other registered adapters share list/search CLI commands with HTTP timeout, retry, and TTL caching. `monid/` provides an opt-in paid gateway (`discover` / `inspect` / `run`) with offline USD/1k pricing reference in `monid/PRICING.md`. `deep_research/` dispatches long-horizon provider-neutral manuscript research (PAID, opt-in). The live registry, not this prose, is authoritative for connector count.

infrastructure.sia (10 files) I

Generic Self-Improving-AI loop utilities: task-layout validation, execution harness, and metric capture reused by `template_sia` (fixture-replay by default).

infrastructure.skills (8 files) I

Discovers SKILL.md frontmatter → .cursor/skill_manifest.json.

Watermark overlays, hashing companions triggered by secure pipeline path.

Invocable tool definitions registered by resource-pool governance; tools mirror `projects/templates/` with git-tracked `templates/*` exemplars.

Rendered-release transmission leaves: LaTeX single-page bookends (`transmission_bookends`), page-check validation for Stage 04 (`transmission_page_check`), barcode strips, figure embedding, and shared models. Consumed by the publishing release workflow; regular package so the documentation module-reference guard resolves nested refs on disk.

`infrastructure.validation` (85 files) I

Markdown + PDF + integrity CLIs underpinning Stage 04 diagnostics.

infrastructure/logrotate.d (0 files) |

Operational templates for deployments (documentation-first; intentionally minimal Python footprint).

Documentation maturity: Coverage statements in Results pull from introspection—not hand-maintained denominators—so newly promoted modules automatically flow into manuscripts after `generate_manuscript_metrics.py`.

FAIR+RSE linkage: MCP-ready `SKILL.md` artefacts align with evaluator heuristics (executability + metadata richness) emphasized by FAIRsoft guidance (Garijo et al. 2024).